# Image Processing project

## Introduction

The objective of this project is to develop a classical computer vision pipeline that can analyze jigsaw puzzle pieces and prepare them for automatic matching and assembly. In Milestone 1, we focus on preprocessing the input images and enhancing their edges so that individual pieces and their boundaries are clearly represented. Using only traditional image processing techniques (without any machine or deep learning), we design and evaluate a sequence of operations that convert the raw puzzle tiles into robust edge maps. These outputs will be used in Milestone 2 to match different pieces together using those edges.

## Pre-Processing

### Description:

The goal of the preprocessing phase is to extract meaningful edges from the puzzle tile images so they can be reliably used in the next stage for piece matching and assembly. In particular, we aim to obtain accurate but not overly detailed edge maps that emphasize the main outlines of objects and puzzle boundaries, while suppressing inner textures and small noisy details that would complicate matching. The preprocessing pipeline is therefore designed to enhance local contrast, reduce noise in flat regions,

and produce clean, stable edges that capture the essential structure of each tile in a way that is robust across different images and lighting conditions.

## Steps:

1)We convert the image into a grayscale image so to prepare it for the edge detection filter.

2)Edge density detection, here we use fixed parameter canny filter for the original image then we count the percentage of white pixels in the result, this gives us a good indicator of how busy the image is this later on helps us base our parameters on the image itself. If the image is too busy we want to blur more and only detect the very strong edges but if it's a simple plain image then we don't want the blurring to be too high and the threshold for the edge detection can be a bit lower.

2)We apply histogram equalization using Clahe (Contrast Limited Adaptive Histogram Equalization) . It differs from normal histogram equalization in the sense that instead of applying to the whole image at once it divides the image into segments and applies histogram equalization on each segment alone so edges and details inside each segment are clearer it performs better than normal adaptive histogram equalization when it comes to flat regions in the image as normal histogram equalization leads to a lot of noise when it comes to these areas, then the enhanced segments are blended smoothly so we can't tell that they were separated.

3)We apply a bilateral filter, to blur the image yet still preserve edges. This is a very important step in our pipeline the bilateral filter works by taking to things into consideration when looking at the nearby pixels in the kernel, intensity and how close they are. If a pixel's intensity is a lot higher than the current pixel's intensity then that likely means it's an edge so the bilateral filter gives that pixel a smaller weight which helps in preserving edges, same happens with closer pixels they are given a higher weight than pixels that are further away from the current pixel in the kernel.

4)We then use an optimized version of canny edge detector to detect the edges in the image called auto canny. The way auto canny works is that for each image we calculate the median from all the pixel values, we choose median over mean so noise pixels aren't considered. The median gives us an indicator at whether the image is dark, bright, or has an average intensity. This way we can determine the values for the lower and upper threshold of the canny edge detector.

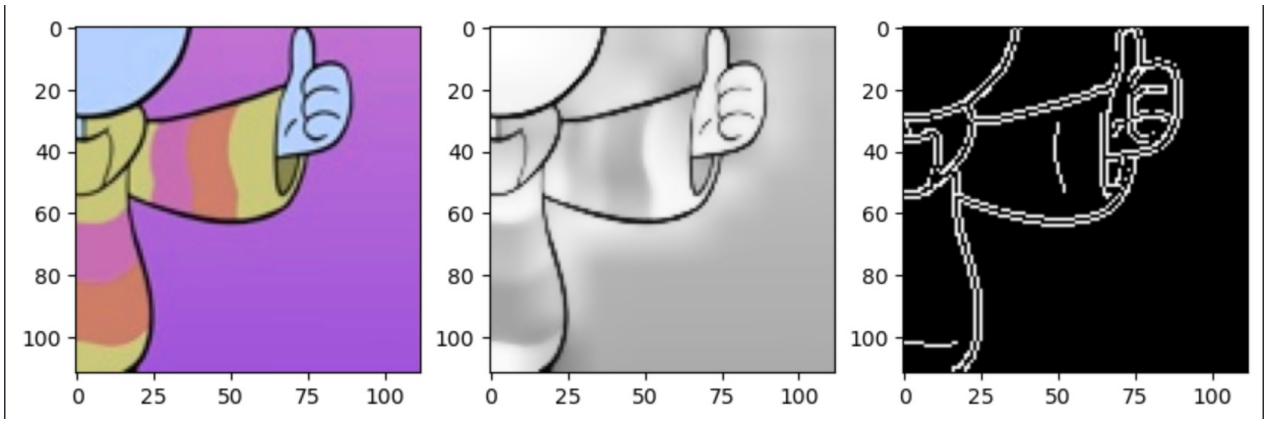
## Image samples:

This pipeline performance varies in the images that we use these are samples of some of the images, the issues that arise in them and some of our solutions to these problems.

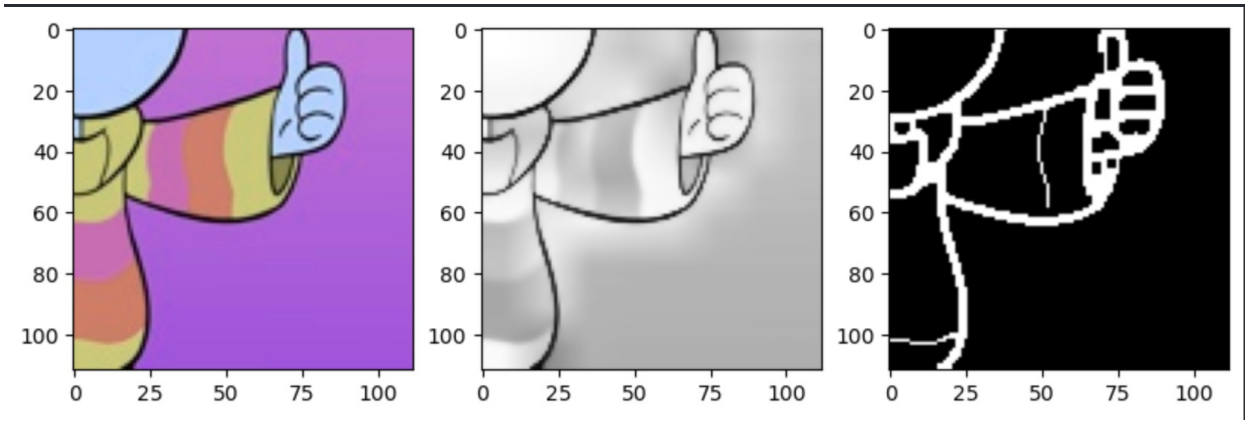
1)Some of the

1) In this image the pipeline successfully extracts the edges from the image , but it also has a problem that double edges appear because of the black stroke in the original image.

We had two solutions to this:

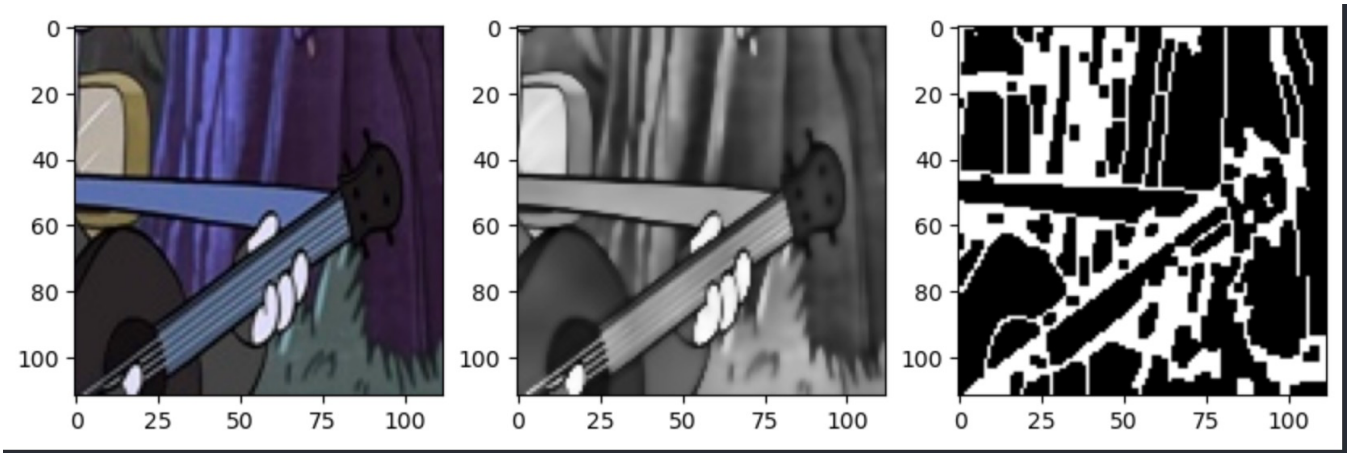



1.1) Apply morphological closing to this result to merge the two edges together.

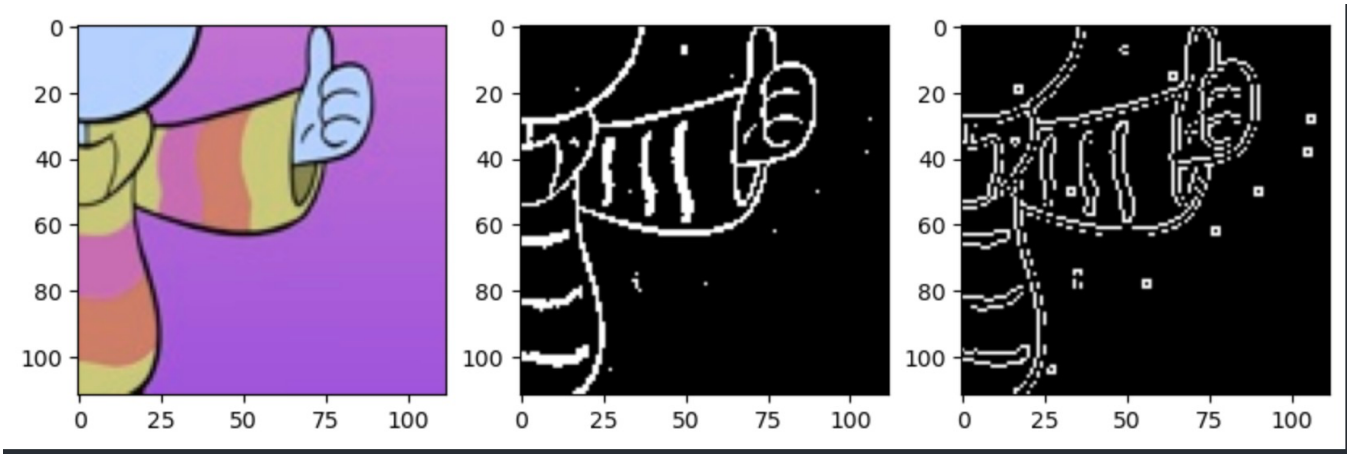



# 3

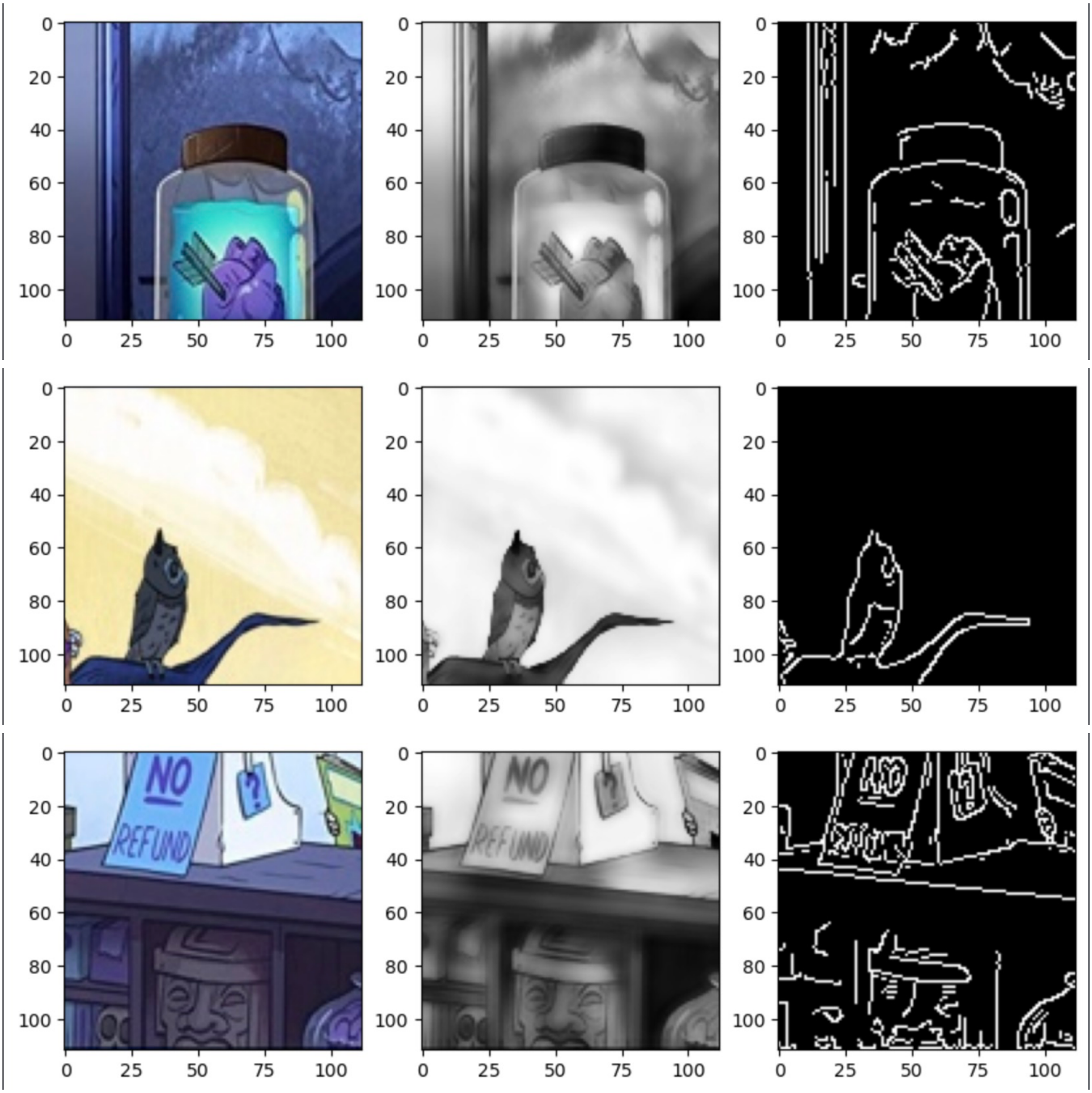
This merges the two edges that were causing the problem but for other images it messes up the edges.

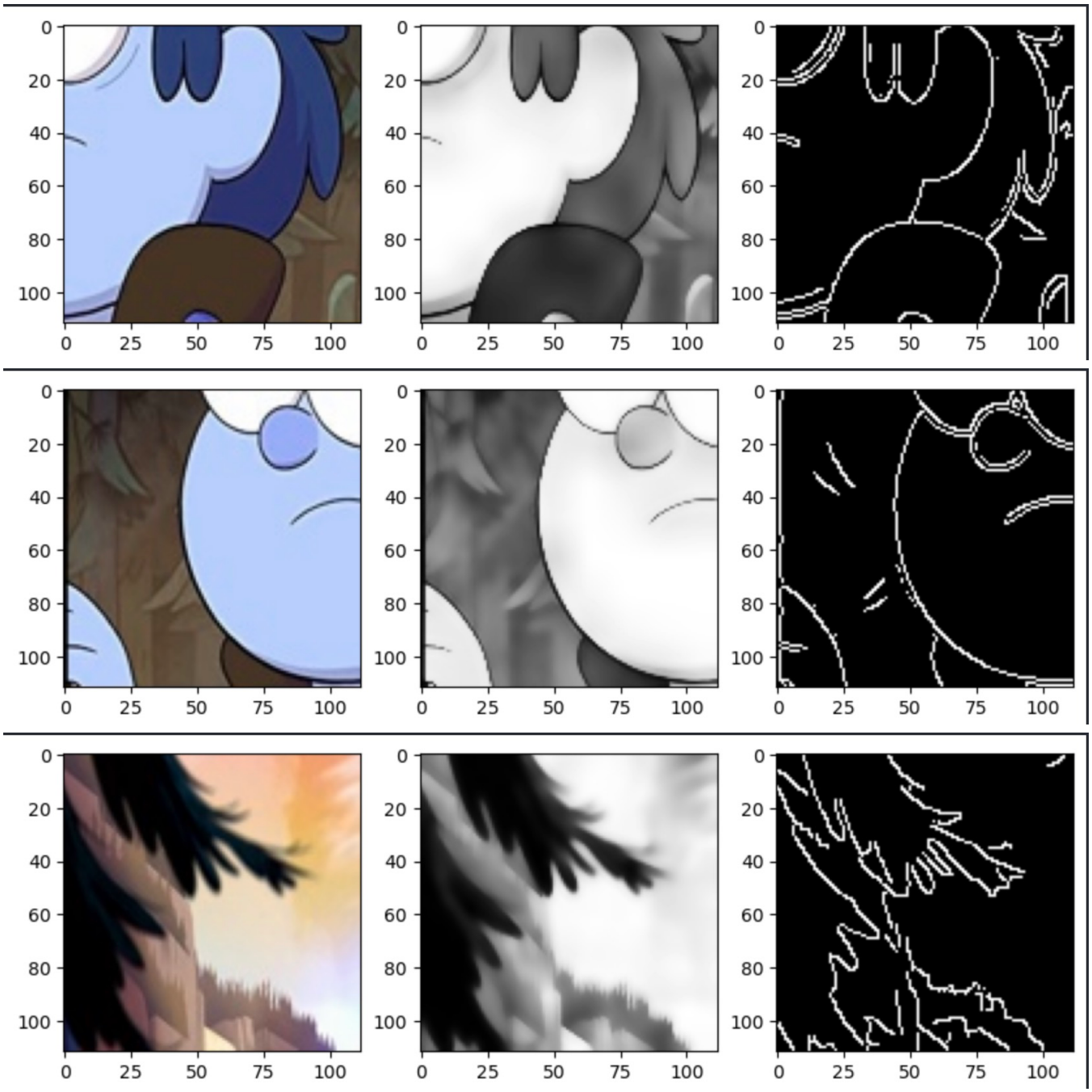



1.2) We also thought of applying thresholding before the canny edge detector to remove the black strokes that caused the double edges in the first place but that messed up the results.

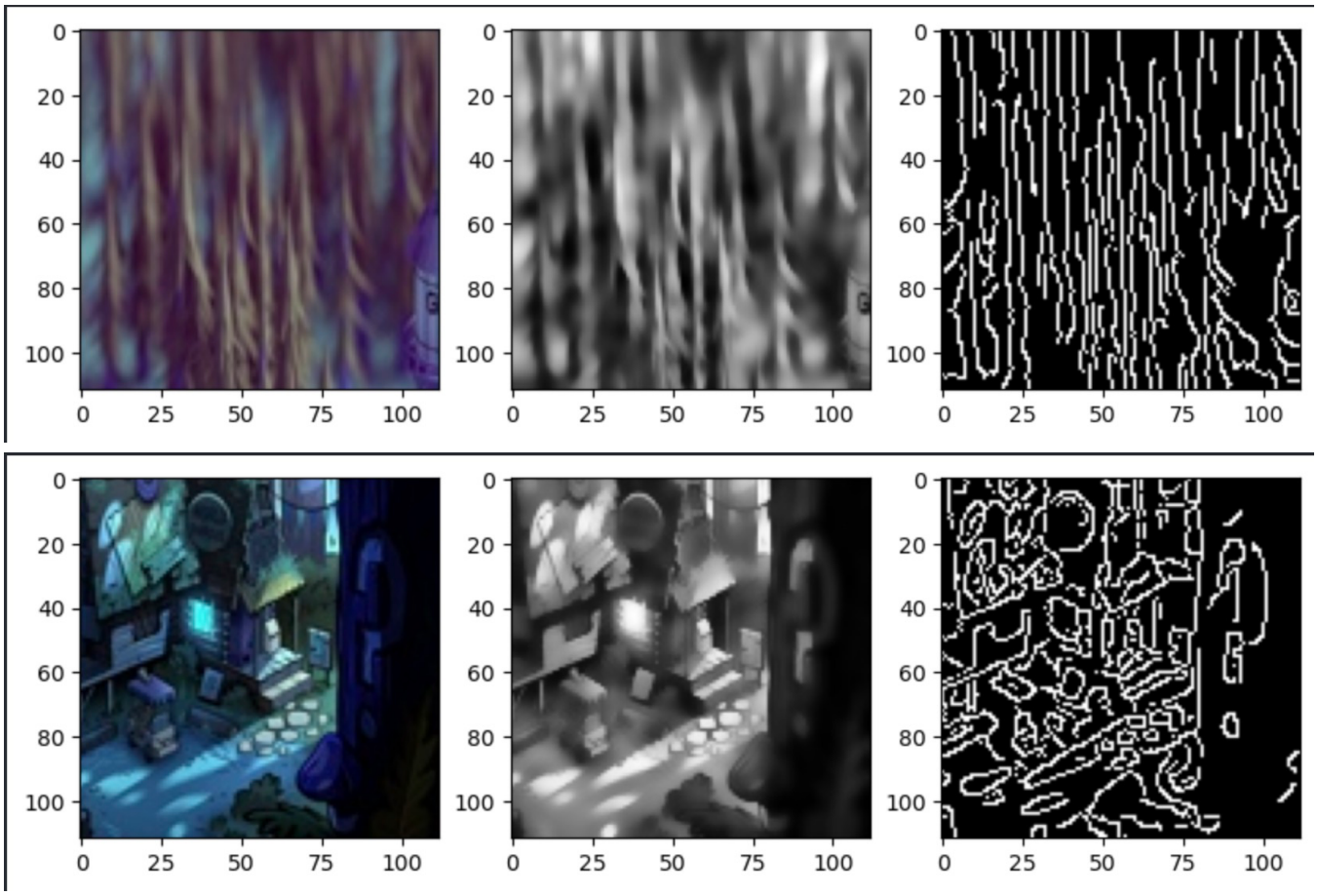


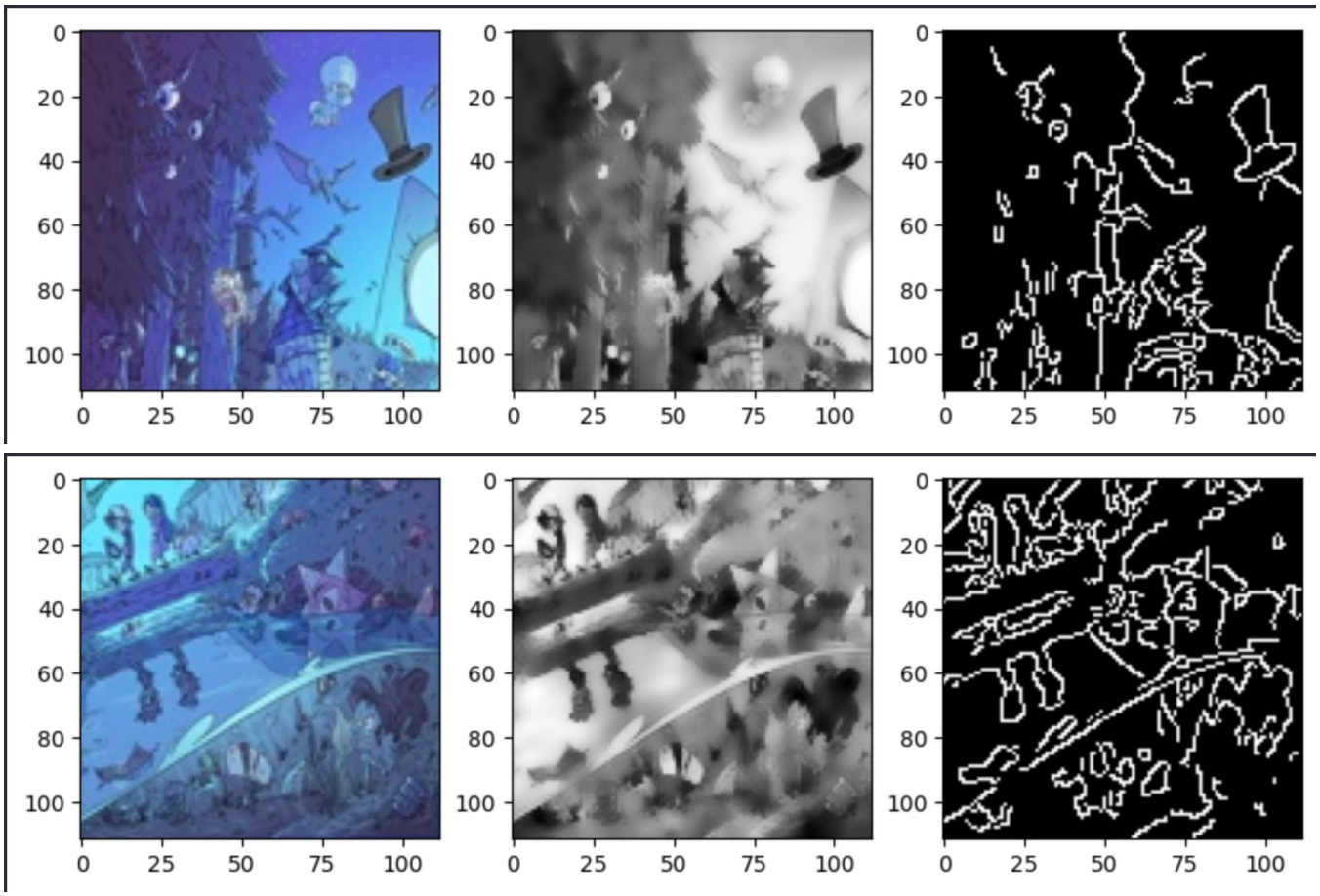
2) Some of the images produced good results without any double edges



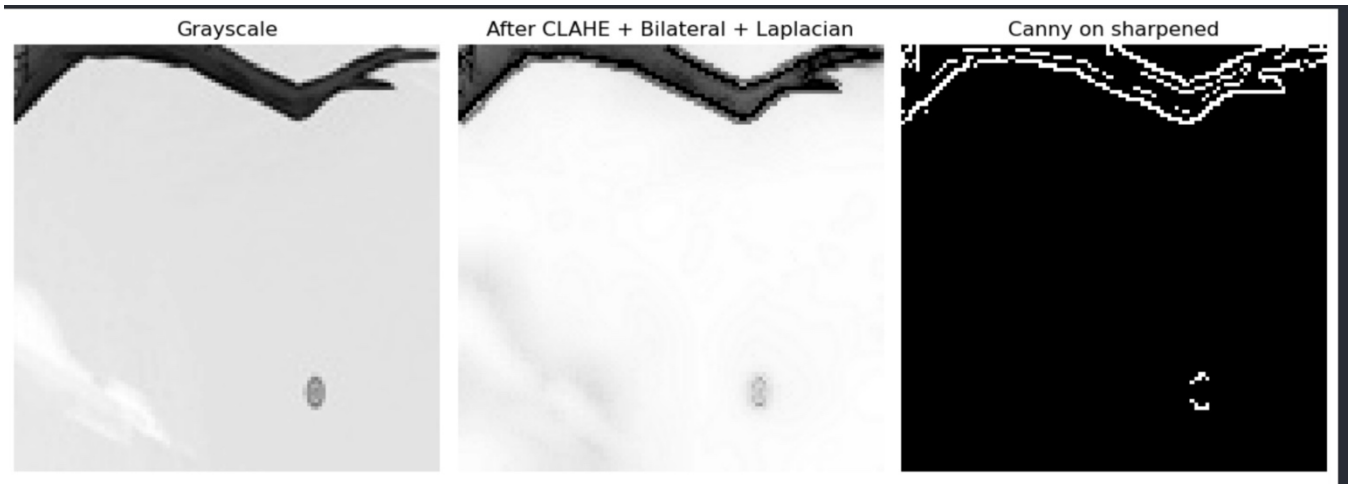

0
20
40
60
80
100
0
25
50
75
100

2) And finally some of the images were just difficult to extract the edges from as they were too detailed:






At some point we also thought about applying sharpening to the images to enhance the edges before they went into the canny edge detector but that caused the output images to have too much noise which is the opposite of what we wanted:



## Segmentation

The project was given to use as folders of different format puzzle pieces 2x2 , 4x4 and 8x8 we segmented the images in these folders manually to produce 4 images from the 2x2 , 16 from the 4x4 and 64 from the 8x8 , we then apply the pipeline to each of them.

The way we did this is that the function that segments the image has the size of the image passed in (2x2 , 4x4, 8x8) and based on it the function can calculate the size of individual puzzle pieces inside the image it gets and split the image at those places to get the individual pieces returning an array of all the individual pieces from the images.

The other approach we tried was to automatically split the image into its 4 segments by detecting the edges between the 4 images and using contours to split the images at those edges. Our pipeline did the following:

As an alternative to manually splitting the puzzle images into 4 sub-tiles using fixed grid coordinates, we initially attempted to detect the boundaries between tiles automatically. In this approach, we first converted the image to grayscale and applied a bilateral filter to suppress noise while preserving edges, then ran Canny edge detection. To emphasize long grid lines, we performed morphological closing with elongated structuring elements in the horizontal and vertical directions, and then applied a probabilistic Hough transform (HoughLinesP) to detect long near-horizontal and near-vertical lines whose midpoints lay close to the image center. The idea was to use these detected lines (and subsequently contours) as the splitting boundaries between the four sub-images. However, in practice the puzzle artwork contained many strong internal edges, which generated numerous spurious line candidates; the Hough detection became highly sensitive to parameter choices, and the true tile boundaries could not be reliably isolated across the dataset. For this reason, we abandoned this automatic splitting method and reverted to a simpler geometric partitioning based on the known 2×2 / 4×4 layout of the input images.

## Conclusion

The conclusion we reached is that it is basically impossible to find a single pipeline that works for all images so we tried to find one that worked for the largest portion of images so that it can be used in phase 2.

# Code

You can find the code in the following github repo : https://github.com/AmirTamer-27/Gravity-Falls-Jigsaw.git

(code is in final branch)

# Phase 2 : Matching

In this phase our goal was to now after segmenting the puzzle pieces and feeding them to our enhancement pipeline match the pieces together and re-assemble the original correct image.

---

## Calculating error:

The main idea of our algorithms is to compare pieces together to find the pieces that match most, what we do is that we extract the top , bottom , left and right row of pixels from each piece which leaves us with 4 strips now for each strip we want to compare it with all its corresponding ones for example all the right strips must be compared with the left strips to find pieces that have similar strips which we can deduce from that these pieces flow into each other and are supposed to be next to each other. We do this by calculating the L1 error(Manhattan distance) which is the summation of the absolute difference between a pixel on strip A and strip B.

## The edge problem:

After our images went through the enhancement pipeline the result was an edge map for each image, when we tried to apply our matching algorithm to these edges we faced a significant problem which is now that most of the image details were gone , now if a border had no edges it's just black so we can't extract any info from it when matching so when we ran the algorithm on the canny results we got 0/110 in all the different image division.

## Our solution:

Since the edges didn't have enough detail to allow us to match the pieces together, we started working on the colored pieces.

Our algorithm after calculating the L1 distance between each strip and its corresponding strips now tried different permutations of putting pieces in all the possible spots and calculating the error that would result from that. Meaning if piece 1 was placed on the top left, all other pieces were tried to be placed next to it and the L1 error between piece 1's right border and the piece that was placed left border was factored in to the total error score. We then found the assembled image that had the best score and that was our result.

After studying the output images from our algorithm we had an observation that some of the images that were re-assembled incorrectly had black lined in the center of the image , which lead us to adding a penalty to the image's cost if the variance of the image's pixels near the center was very high this along with a technique called deep center lock which looks at each corner of the re-assembled image's pieces and makes sure that there are no sudden changes vertically horizontally and diagonally adding the result in the cost improved our algorithm's performance very much.

## Performance issue:

The approach we had at that point worked great for 2x2 puzzles but since it was based on permutations it would take 20 trillion permutations to run on the 4x4 puzzle and 127 octovigintillion (I didn't actually now that was a number) permutations on the 8x8 puzzle which is computationally impossible so we had to think of another approach to match the pieces.

## Greedy approach and Prims:

Our initial approach was instead of trying all permutations of the puzzle pieces we would start on the top left of the puzzle with a random piece then for that piece we would find the best match for its right border using the pre-calculated L1 distances and add that piece after it, we keep moving like this row by row until we fill the whole puzzle, we try every single piece we have as a starting point and store its total MSE the best starting point's MSE is what we consider the correct orientation.

The greedy approach gave us good results but upon research we found another algorithm called prim's that we can use it works almost the same way as the greedy approach but instead of only moving row by row when we place a piece at the top left we look at all its possible borders , so for top left we look at right and bottom , and then we find the best piece for right and best piece for bottom using the pre-calculated values, the piece with the lower MSE is the one that is added to the puzzle. This gave us better results than the normal greedy approach.

## Histograms of oriented gradients:

We tried to find a way to integrate phase 1 with phase 2. Since we were only using colors up until now, we were missing out on the flow of edges thought the pieces and for the reasons stated above using our edge maps only made things worse. After some research we found out about histograms of oriented gradients. A histogram of oriented gradients takes an image and for each pixel it moves using a small 1x1 filter [-1,0,1] in doing so it calculates the gradient vector for each pixel then takes the edge orientation and places it in a histogram essentially zipping all the edge information in that image into a histogram that represents how the edges flow in the image.

After factoring HOGs into our algorithm when calculating costs between strips it improved giving us an extra 5-10 images correct across the different puzzle divisions.

## Desperation locking:

Upon looking for ways to improve our algorithm we found two techniques called desperation locking. The way desperation locking works is that for every strip after the distances are calculated it checks to see, for this specific strip what is the best match and what is the second-best match if there is a large difference between them that means that this strip can ONLY be matched with that one so we force the cost between them to be a very low number.

This solves the problem of ambiguity in matching by prioritizing relative distinctiveness over simple absolute scores. In puzzle solving, a low error score usually indicates a good match, but sometimes multiple pieces yield very similar scores, creating uncertainty where noise could lead to an incorrect choice using desperation locking leads to effectively forcing the global solver to anchor the entire solution around these highly certain matches and preventing early errors from propagating.

## The Blue-sky problem:

The algorithm we had at this point was working very well for 2x2 puzzles and 8x8 puzzles getting 106/110 puzzles correct in 2x2 and 95 puzzles correct for the 4x4, however it wasn't performing well in the 8x8 puzzle getting us only 18-20 correct puzzles. After doing some research we found that the issue was in a problem known as the blue-sky problem. The blue-sky problem is defined as the following, if we have a picture of a blue sky and we divide it into smaller pieces all of them will have great scores together, and some pieces may have better scores with incorrect matches than their correct matches, this is what was happening when we divided our image into 8x8 pieces.

The Desperation locking works on finding a piece that matches perfectly with another. However, this often fails in puzzles with repetitive or low-detail regions, such as blue skies or grassy fields (the "Blue Sky Problem"). In these areas, a piece might match several neighbors almost equally well. Desperation locking sees a high-quality match and locks it in prematurely, even though it could be very wrong which causes it to mess up the whole puzzle afterwards when using the greedy approach since it uses the hacked matrix that desperation locking made.

The solution we found was a uniqueness constraint on prims algorithm itself, now instead of choosing the piece that gives us the lowest possible error from all the pieces that could match we chose the most unique piece meaning the piece that has the biggest difference between the first and second match for our original piece. This gave us better results for the 8x8 now getting around 55 images correctly out of 110.

However, this uniqueness approach didn't quite work with the 4x4 and the 2x2 since the blue-sky problem rarely appears there, so we handled them differently 4x4 and 2x2 work with the desperation locking and normal prims while 8x8 works with prims alongside the uniqueness constraint.